Melissa Garcia Ospina

Mar. 11, 2026

Foundations of Programming: Python

Assignment 07

GitHub Link

Classes & Objects

Introduction

This document explains the essential tools used for programming, such as, how a code is structured (statements, functions, classes), roles of different types of classes (data, presentation, processing), foundation for object-oriented programming (OOP) (constructors, attributes, properties). These tools help programmers develop structured programs into reusable and logical codes that can be used effectively.

Implementation of Tools

Functions allow for reusable behavior with parameters as placeholders and arguments as supplier values. Classes combines state and behavior, unlike functions. Statements are organized into functions when they are being used throughout the program. There can be many statements, functions and classes used in a code. Variables and constants in a class are called fields, attributes or properties and functions in a class are called methods, depending how these are all defined and used (Mod07-Notes) (external link).

Each object that is created from a class holds its own set of data, also known as data encapsulation and it is crucial to object-oriented programming. This allows one to work with multiple instances of the same class independently. Overall, the class defines the program where the object is in, however, it does not define how each object behaves. Abstraction focuses on what an object is intended to do, while encapsulation focuses on how it gets done.

Classes can focus on processing, data or presentation. Data classes usually have attributes, constructors, properties and methods. While processing and presentation classes only have methods. Classes depend on the following:

- Attributes are data that is associated with an object. They are considered variables storing or describing information or “state” about an object. The data can be integers, strings or custom type.
- Constructor is a method (function) that is called automatically when an object of a class is made, meaning one does not have to call the constructor out. It sets up any initial data (state) of the

object. This is the initialization of the object's attributes. They are named __init__. It is best programming practice to define the constructors as it would alleviate the default one with no parameters.

- The keyword "self" that is used in the constructor method refers to the data or functions found in an object instance, not in the class itself. Class code only loads into memory once, it can also have a "copy" of itself, and these are identified by using the "self". This is shown in Figure 1.
- Private attributes are used when once wants the attributes to not be changed by code outside of the class. This means it is controlled by a method, such as a constructor.
- Properties manage attribute data. For each attribute, one should create two properties, one "gets" data and the other "sets" the data. As shown in Figure 1. It is good practice top know that the setter decorator and function name must match.

```
@property  3 usages (1 dynamic)
def course_name(self):
    return self.__course_name.title()

@course_name.setter  2 usages (1 dynamic)
def course_name(self, value: str):
    self.__course_name = value

def __str__(self):
    return f"{self.first_name},{self.last_name},{self.course_name}"
```

Figure 1. "Self" and Properties Example

Inherited Code is used in programming when a new class inherits data and behaviors (properties and methods) from an existing class. This can also be called Base and Derived Class. There is the __int__() constructor and the __str__() to string method that are used as pythons "object class"; which is the top-level class that forms the basis of the class hierarchy (Mod07-Notes) (external link). With the string method, one can easily control the string representation of an object. These are shown in Figure 2.

```python
class Student(Person):  2 usages
    """
    A class representing student data.

    Properties:
    first_name (str): Student's first name.
    last_name (str): Student's last name.

    ChangeLog:
    MelissaGO, 3/10/2026, Created the class.
    """
    def __init__(self, first_name: str = "", last_name: str = "", course_name: str = ""):
        super().__init__(first_name=first_name,last_name=last_name)
        self.course_name = course_name

    @property  3 usages (1 dynamic)
    def course_name(self):
        return self.__course_name.title()

    @course_name.setter  2 usages (1 dynamic)
    def course_name(self, value: str):
        self.__course_name = value

    def __str__(self):
        return f"{self.first_name},{self.last_name},{self.course_name}"
```

Figure 2. Inherited Code Example

Creating the code

I created a python code that demonstrated the use of constants, variables and print statements, while incorporating set of data classes to display a message about a student's registration for a python course. This is shown throughout Figures 1 through 3.

```python
class Person:  1 usage
    Properties:
    first_name (str): The person's first name.
    last_name (str): The person's last name.

    ChangeLog:
    MelissaGO, 3/10/2026, Created the class.
    """

    def __init__(self, first_name: str = "", last_name: str = ""):
        self.first_name = first_name
        self.last_name = last_name

    @property  3 usages (1 dynamic)
    def first_name(self):
        return self.__first_name.title()

    @first_name.setter  2 usages (1 dynamic)
    def first_name(self, value: str):
        if value.isalpha() or value == "":
            self.__first_name = value
        else:
            raise ValueError ("The first name should not contain numbers.")

    @property  3 usages (1 dynamic)
    def last_name(self):
        return self.__last_name.title()

    @last_name.setter  2 usages (1 dynamic)
    def last_name(self, value: str):
        if value.isalpha() or value == "":
            self.__last_name = value
        else:
            raise ValueError ("The last name should not contain numbers.")
```

Figure 3. Code Example

Summary

Understanding how different structural elements are used, is essential for programming. Statements are single executable instructions, functions group statements together. Classes act as blueprint for objects, data classes store the state, presentation classes deal with the displayed information and processing classes perform the logic. Classes depend on constructor that starts a new object when created, attributes that store the data within an object and properties that allow access to the attributes. Overall, these concepts are an important foundation for developing a structured and reliable code.